

Prácticas de Lógica Computacional

Introducción manejo Listas Prolog

Erik Figueiral Alonso

erik.figueiral.alonso@uvigo.gal

SWI-Prolog · 3.º de Ingeniería Informática

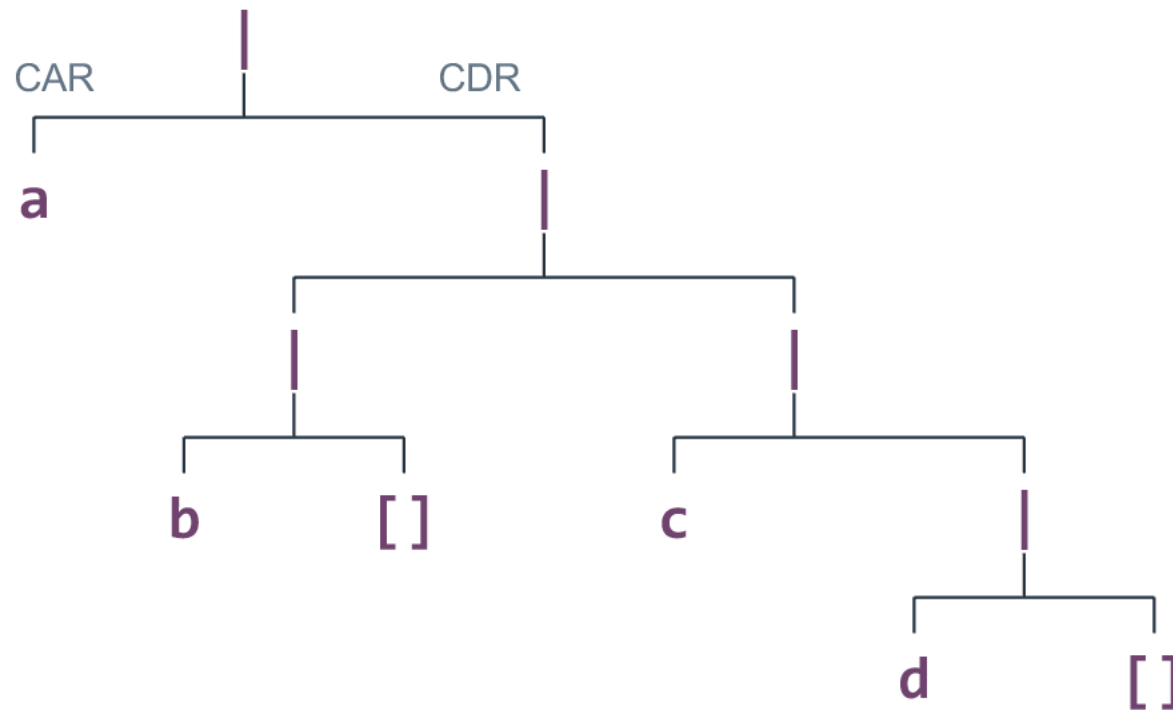
Índice

1 Representación

2 Ejercicios

Cada célula tiene cabeza y cola

`[a, [b], c, d]`



CAR · cabeza

`a`

CDR · cola

`[[b], c, d]`

Cada `[]` marca el final de una lista.

$\text{CADR} = [b] \cdot \text{CADDR} = c$

Referencia: capítulo 10, figura 10.1 · CAR y CDR: cabeza y cola

Listas: elementos ordenados entre corchetes

Una lista conserva el orden y puede contener otras listas.

`[a,[b],c,d]` % cuatro elementos; `[b]` es uno de ellos

La misma lista	Cómo leerla
<code>[a,b,c]</code>	Tres elementos separados por comas
<code>[a [b,c]]</code>	Cabeza: <code>a</code> ; cola: <code>[b,c]</code>
<code>[a,b [c]]</code>	Dos elementos iniciales; cola: <code>[c]</code>
<code>[a,b,c []]</code>	La cola final es la lista vacía

Sin coma antes de `|`. `[]` no tiene elementos; `[a,b,c,[]]` tiene cuatro.

La variable anónima: lo que no necesitamos

Consola de SWI-Prolog

```
?- [Cabeza|Cola] = [a,b,c].  
Cabeza = a, Cola = [b,c].
```

```
?- [_,X|_] = [a,b,c].  
X = b.
```

Cada `_` es una variable distinta: no necesitamos conocer su valor.
Una variable con nombre usada una sola vez puede dar un aviso singleton.

Usa `_` si ese dato no interesa; revisa si el aviso revela una errata.

Archivo listas.pl: cláusulas. Consola: `consult(listas).` y consultas con `?-`

Ejercicio: primero/2

Sintaxis	Semántica	Firma
primero(L,X)	X es el primer elemento de L. L debe tener al menos un elemento.	primero/2

Consulta en consola	Resultado esperado
?- primero([1,2,3],X).	X = 1.
?- primero([1,2,3],2).	false.
?- primero([],X).	false.

Solución: primero/2

Archivo listas.pl

```
primero([Car|_],Car).
```

El mismo Car une la cabeza de L con el segundo argumento. Accedemos a la cabeza directamente: $O(1)$.

Consulta en consola	Respuesta
?- primero([1,2,3],X).	X = 1.
?- primero([1,2,3],2).	false.
?- primero([],X).	false.

Ejercicio: segundo/2

Sintaxis	Semántica	Firma
segundo(L,X)	X es el segundo elemento de L. L debe tener al menos dos elementos.	segundo/2

Consulta en consola	Resultado esperado
?- segundo([a,b,c],X).	X = b.
?- segundo([a],X).	false.

Solución: segundo/2

Archivo listas.pl

```
segundo( [_ , Cadr | _] , Cadr ) .
```

Equivale a
segundo([_|[Cadr|_]] , Cadr).
CADR: cabeza de la cola.

Consulta en consola	Respuesta
?- segundo([a,b,c],X).	X = b.
?- segundo([a],X).	false.

Ejercicio: ultimo/2

Sintaxis	Semántica	Firma
ultimo(L,X)	X es el último elemento de L. L debe ser una lista no vacía.	ultimo/2

Consulta en consola	Resultado esperado
?- ultimo([a,b,c],X).	X = c ; false.
?- ultimo([3],X).	X = 3 ; false.
?- ultimo([],X).	false.

Solución: ultimo/2

Archivo listas.pl

```
ultimo([X],X).
ultimo([_|Cola],X) :-
    ultimo(Cola,X).
```

Se descarta una cabeza en cada llamada.
La lista de un elemento satisface el axioma.

Seguimiento de la consulta

```
ultimo([a,b,c],X)
      |
ultimo([b,c],X)
      |
ultimo([c],X)
      |
X = c
```

Consulta en consola	Respuesta
?- ultimo([a,b,c],X).	X = c ; false.
?- ultimo([3],X).	X = 3 ; false.
?- ultimo([],X).	false.

Solución: una respuesta representa muchas listas

Consola de SWI-Prolog

```
?- ultimo(L,3).  
L = [3] ;  
L = [_5576,3] ;  
L = [_5576,_5582,3] ;  
...
```

`_5576` es una variable todavía libre.
Su nombre lo asigna el intérprete
y puede cambiar entre ejecuciones.

`[_5576,3]` admite, por ejemplo:
`[a,3]`, `[7,3]` o `[[a,b],3]`.

Cualquier lista finita no vacía que termine en 3.

Cada ; aumenta las posibilidades; la enumeración completa no termina.

Ejercicio: penultimo/2

Sintaxis	Semántica	Firma
penultimo(L,X)	X es el penúltimo elemento de L. L debe tener al menos dos elementos.	penultimo/2

Consulta en consola	Resultado esperado
?- penultimo([a,b,c],X).	X = b ; false.
?- penultimo([a,b],X).	X = a ; false.
?- penultimo([a],X).	false.

Solución: penultimo/2

Archivo listas.pl

```
penultimo([X,_],X).  
penultimo([_|Cola],X) :-  
    penultimo(Cola,X).
```

Base: quedan exactamente dos elementos.

Si avanzamos por la cola, el penúltimo sigue siendo el mismo.

Consulta en consola	Respuesta
?- penultimo([a,b,c],X).	X = b ; false.
?- penultimo([a,b],X).	X = a ; false.
?- penultimo([a],X).	false.

Ejercicio: izqda_der/3

Sintaxis	Semántica	Firma
izqda_der(I,D,L)	I está inmediatamente a la izquierda de D en L. Por ejemplo, izqda_der(1,2,[1,2,3]) es cierto.	izqda_der/3

Consulta en consola	Resultado esperado
?- izqda_der(2,3,[1,2,3]).	true ; false.
?- izqda_der(1,3,[1,2,3]).	false.
?- izqda_der(I,D,[1,2,3]).	I = 1, D = 2 ; I = 2, D = 3 ; false.

Solución: izqda_der/3

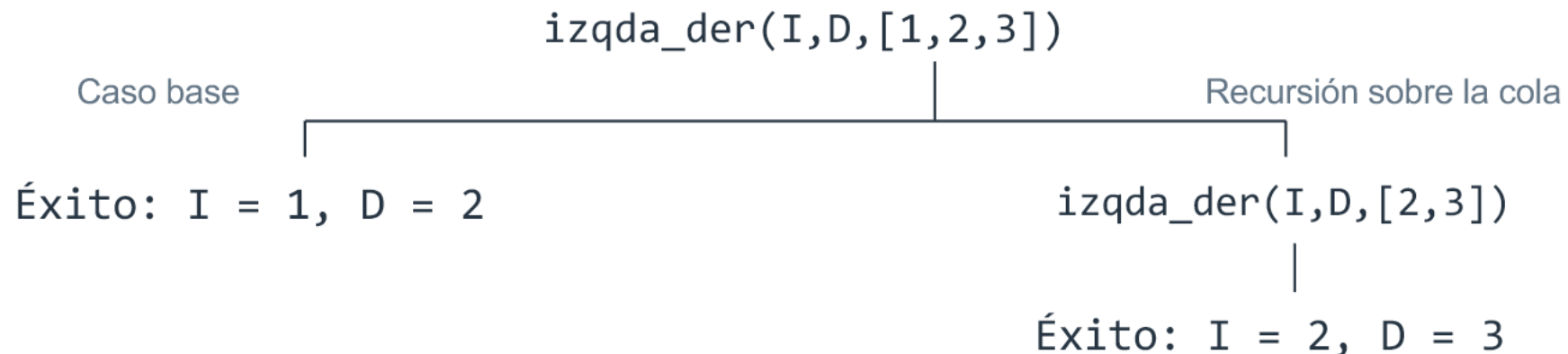
Archivo listas.pl

```
izqda_der(I,D,[I,D|_]).
izqda_der(I,D,[_|Cdr]) :-
    izqda_der(I,D,Cdr).
```

```
?- izqda_der(I,D,[1,2,3]).
```

Consola de SWI-Prolog

```
?- izqda_der(2,3,[1,2,3]).
true ; false.
?- izqda_der(1,3,[1,2,3]).
false.
```



La cola [3] ya no contiene dos vecinos: no hay más soluciones.

Ejercicio: derecha_izq/3

Sintaxis	Semántica	Firma
derecha_izq(D,I,L)	D está inmediatamente a la derecha de I en L. El primer argumento es el elemento de la derecha.	derecha_izq/3

Consulta en consola	Resultado esperado
?- derecha_izq(2,1,[1,2,3]).	true ; false.
?- derecha_izq(1,2,[1,2,3]).	false.

Solución: derecha_izq/3

Archivo listas.pl · alternativa A

```
derecha_izq(D,I,[I,D|_]).  
derecha_izq(D,I,[_|Cdr]) :-  
    derecha_izq(D,I,Cdr).
```

Archivo listas.pl · alternativa B

```
derecha_izq(D,I,L) :- izqda_der(I,D,L).
```

Consulta en consola	Respuesta
?- derecha_izq(2,1,[1,2,3]).	true ; false.
?- derecha_izq(1,2,[1,2,3]).	false.

Elegid una definición: si cargáis ambas, repetiréis soluciones.

Ejercicio: contigua/3

Sintaxis	Semántica	Firma
contigua(X,Y,L)	X e Y ocupan posiciones consecutivas en L, en cualquiera de los dos órdenes.	contigua/3

Consulta en consola	Resultado esperado
?- contigua(b,a,[a,b,c]).	true ; false.
?- contigua(a,c,[a,b,c]).	false.

Solución: contigua/3

Archivo listas.pl

```
contigua(X,Y,L) :- izqda_der(X,Y,L).  
contigua(X,Y,L) :- derecha_izq(X,Y,L).
```

X está justo a la izquierda de Y o justo a su derecha.
Reutilizamos las dos relaciones que ya conocemos.

Consulta en consola	Respuesta
?- contigua(b,a,[a,b,c]).	true ; false.
?- contigua(a,c,[a,b,c]).	false.

Ejercicio: intercalar/3

Sintaxis	Semántica	Firma
intercalar(A,B,R)	R alterna elementos de A y B, empezando por A. Si A acaba, paramos; si B acaba, tomamos solo uno más de A.	intercalar/3

Consulta en consola	Resultado esperado
?- intercalar([1,2],[a],R).	R = [1,a,2].
?- intercalar([1],[a,b],R).	R = [1,a].
?- intercalar([1,2],[],R).	R = [1].
?- intercalar([], [a], R).	R = [].

Solución: intercalar/3

Archivo listas.pl

```
intercalar([],_,[]).
intercalar([X|_],[],[X]).
intercalar([X|Xs],[Y|Ys],[X,Y|R]) :-
    intercalar(Xs,Ys,R).
```

Cada paso aporta un elemento de A y otro de B.

Los casos base fijan cuándo dejamos de recoger elementos.

Consulta en consola	Respuesta
?- intercalar([1,2],[a],R).	R = [1,a,2].
?- intercalar([1],[a,b],R).	R = [1,a].
?- intercalar([1,2],[],R).	R = [1].
?- intercalar([], [a], R).	R = [].

Ejercicio: select/3

Sintaxis	Semántica	Firma
select(X,L,R)	R se obtiene eliminando una aparición de X en L. El resto conserva su orden.	select/3

Consulta en consola	Resultado esperado
?- select(a,[a,b,a],R).	R = [b,a] ; R = [a,b] ; false.
?- select(x,[a,b],R).	false.

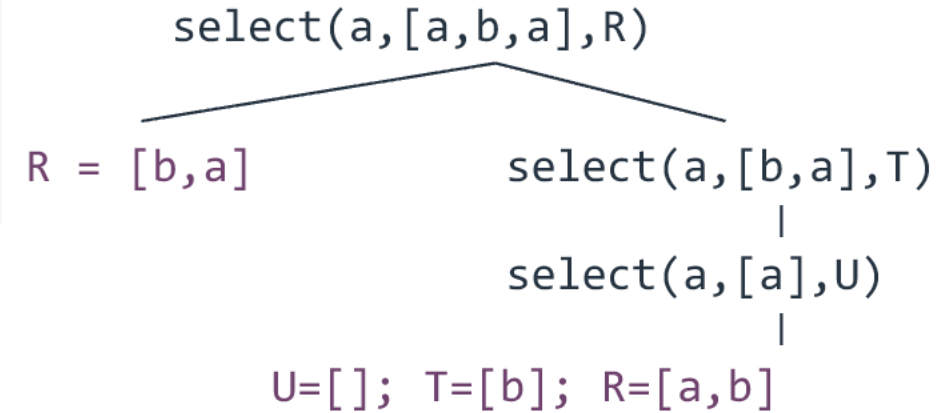
Solución: select/3

Archivo listas.pl

```
select(X,[X|R],R).
select(X,[Y|L],[Y|R]) :-
    select(X,L,R).
```

Primera rama: se elimina el primer a.
La otra rama conserva a y b hasta eliminar la segunda aparición de a.

Seguimiento de la consulta



Consulta en consola	Respuesta
?- select(a,[a,b,a],R).	R = [b,a] ; R = [a,b] ; false.
?- select(x,[a,b],R).	false.

Ejercicio: miembro/2

Sintaxis	Semántica	Firma
miembro(X,L)	X ocupa alguna posición de L. Cada aparición puede aportar una solución.	miembro/2

Consulta en consola	Resultado esperado
?- miembro(X,[a,b]).	X = a ; X = b ; false.
?- miembro(3,[3,a,3,b,3,c,4]).	true ; true ; true ; false.

SWI-Prolog ya ofrece member/2. Aquí implementamos miembro/2 para practicar.

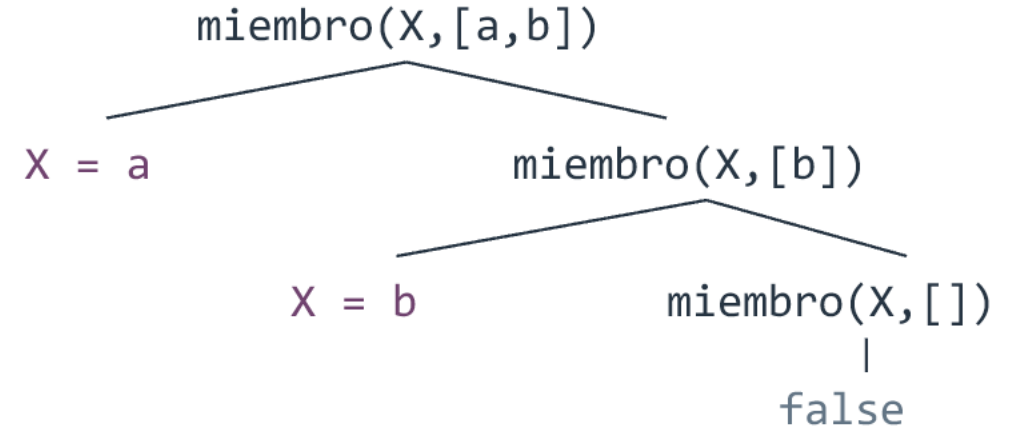
Solución: miembro/2

Archivo listas.pl

```
miembro(X,[X|_]).
miembro(X,[_|Cola]) :-
    miembro(X,Cola).
```

El axioma encuentra la cabeza.
Al pedir ;, la regla busca en la cola.
Cada coincidencia da una respuesta.

Seguimiento de la consulta



Consulta en consola	Respuesta
?- miembro(X,[a,b]).	X = a ; X = b ; false.
?- miembro(3,[3,a,3,b,3,c,4]).	true ; true ; true ; false.

Ejercicio: concat/3

Sintaxis	Semántica	Firma
concat(A,B,C)	C contiene los elementos de A seguidos de los de B. Se conserva el orden de ambas listas.	concat/3

Consulta en consola	Resultado esperado
?- concat([a,b],[c],R).	R = [a,b,c].
?- concat(A,B,[a,b]).	A = [], B = [a,b] ; A = [a], B = [b] ; A = [a,b], B = [].

SWI-Prolog ya ofrece append/3. Aquí implementamos concat/3 para practicar.

Solución: concat/3

Archivo listas.pl

```
concat([ ],B,B).
concat([X|A],B,[X|C]) :-
    concat(A,B,C).
```

Con A vacía, el resultado es B.
Con cabeza X, C conserva X y se concatenan las colas.

Consulta en consola	Respuesta
?- concat([a,b],[c],R).	R = [a,b,c].
?- concat(A,B,[a,b]).	A = [], B = [a,b] ; A = [a], B = [b] ; A = [a,b], B = [].

Concat: recorrer la primera lista y reconstruir

Consola de SWI-Prolog

```
?- concat([1,2,3],[a,b],R).
```

Primera lista	Segunda lista	Resultado pendiente
[1 [2,3]]	[a,b]	R = [1 R1]
[2 [3]]	[a,b]	R1 = [2 R2]
[3 []]	[a,b]	R2 = [3 R3]
[]	[a,b]	R3 = [a,b]

Al volver: $R2 = [3,a,b]$ · $R1 = [2,3,a,b]$

```
R = [1,2,3,a,b].
```

Ejercicio: inv/2

Sintaxis	Semántica	Firma
inv(L,R)	R contiene los elementos de L en orden inverso. Trabajamos con L conocida y finita.	inv/2

Consulta en consola	Resultado esperado
?- inv([a,b,c],R).	R = [c,b,a].
?- inv([],R).	R = [].

Solución: inv/2

Archivo listas.pl

```
inv([], []).
inv([X|Cola], R) :-
    inv(Cola, Inversa),
    concat(Inversa, [X], R).
```

Invertir la cola y añadir la antigua cabeza al final.
Esta versión sencilla recorre listas varias veces: $O(n^2)$.

Consulta en consola	Respuesta
?- inv([a,b,c],R).	R = [c,b,a].
?- inv([],R).	R = [].

Inv: llamadas recursivas y concatenaciones

Consola de SWI-Prolog

```
?- inv([1,2,3],R).
```

Lista que se invierte	Llamada a inv/2	Concatenación pendiente
[1 [2,3]]	inv([2,3],R1)	concat(R1,[1],R)
[2 [3]]	inv([3],R2)	concat(R2,[2],R1)
[3 []]	inv([],R3)	concat(R3,[3],R2)
[]	Axioma: R3 = []	No se añade concatenación

Al volver de la recursión

R2 = [3] R1 = [3,2] R = [3,2,1]

En el último paso: concat([3,2],[1],R).

Ejercicio: aplanar/2

Sintaxis	Semántica	Firma
aplanar(L,R)	R contiene las hojas de L en su orden original. L: lista finita anidada de átomos y números, sin variables.	aplanar/2

Consulta en consola	Resultado esperado
?- aplanar([a,[b],[c,[d]]],R).	R = [a,b,c,d] ; false.
?- aplanar([[],[]],R).	R = [] ; false.

Aplanar: primero, los casos base

Casos base

```
aplanar([],[]).
```

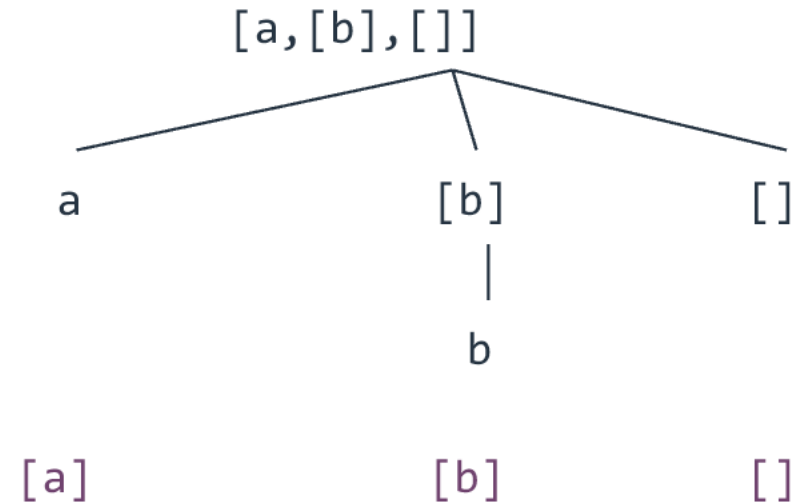
La lista vacía no aporta elementos.

```
aplanar(X,[X]) :-  
    atomic(X), X \== [].
```

Un dato atómico aporta una lista unitaria.
La comprobación excluye []: `atomic([])`
también tiene éxito.

Concatenar `[a]`, `[b]` y `[]` produce `[a,b]`.

Descomposición de `[a,[b],[]]`



Resultados de los casos base

Solución: aplanar/2

Archivo listas.pl

```
aplanar([],[]).  
aplanar(X,[X]) :- atomic(X), X \== [].  
aplanar([C|Cs],R) :-  
    aplanar(C,A), aplanar(Cs,B),  
    concat(A,B,R).
```

Consola de SWI-Prolog

```
?- aplanar([a,[b],[],[c,[d]]],R).  
R = [a,b,c,d] ; false.  
  
?- aplanar([],[]),R.  
R = [] ; false.
```

atomic([]) también es cierto; X \== [] evita conservarlo como hoja.

Ejercicio: subconjunto/2

Sintaxis	Semántica	Firma
subconjunto(S,C)	Todos los elementos de S pertenecen a C. S y C son listas sin elementos repetidos.	subconjunto/2

Consulta en consola	Resultado esperado
?- subconjunto([b,a],[a,b,c]).	true ; false.
?- subconjunto([], [a,b]).	true.
?- subconjunto([d],[a,b,c]).	false.

Desde este ejercicio trabajamos con conjuntos:
las listas de entrada no contienen elementos repetidos.

Solución: subconjunto/2

Archivo listas.pl

```
subconjunto([],_).  
subconjunto([X|Xs],C) :-  
    select(X,C,Resto),  
    subconjunto(Xs,Resto).
```

S y C son conjuntos sin repeticiones.
select/3 retira de C el elemento que ya se ha comprobado.

Consulta en consola	Respuesta
?- subconjunto([b,a],[a,b,c]).	true ; false.
?- subconjunto([],[a,b]).	true.
?- subconjunto([d],[a,b,c]).	false.

Ejercicio: linea/3

Sintaxis	Semántica	Firma
linea(X,C,R)	R contiene un par [X,Y] por cada Y de C. C es un conjunto: lista sin elementos repetidos.	linea/3

Consulta en consola	Resultado esperado
?- linea(a,[1,2],R).	R = [[a,1],[a,2]].
?- linea(a,[],R).	R = [].

Solución: linea/3

Archivo listas.pl

```
linea(_, [], []).  
linea(Elemento, [Car | Cdr],  
      [[Elemento, Car] | Resto]) :-  
    linea(Elemento, Cdr, Resto).
```

X se combina con cada elemento del conjunto C.
Cada par [X,Y] es una lista de dos elementos.

Consulta en consola	Respuesta
?- linea(a,[1,2],R).	R = [[a,1],[a,2]].
?- linea(a,[],R).	R = [].

Linea: construir el resultado paso a paso

Consola de SWI-Prolog

```
?- linea(1,[a,b],R).
```

Elemento	Lista	Resultado pendiente	Llamada recursiva
1	[a [b]]	$R = [[1,a] R1]$	<code>linea(1,[b],R1)</code>
1	[b []]	$R1 = [[1,b] R2]$	<code>linea(1,[],R2)</code>
1	[]	$R2 = []$	Axioma

Al volver de la recursión

```
R1 = [[1,b]|[]] = [[1,b]]
R  = [[1,a]|[[1,b]]] = [[1,a],[1,b]]
```

El elemento 1 se mantiene. Cada llamada añade un par [1,Car].

Ejercicio: cartesiano/3

Sintaxis	Semántica	Firma
<code>cartesiano(A,B,R)</code>	R contiene todos los pares $[X,Y]$, con X en A e Y en B. A y B son listas sin elementos repetidos.	<code>cartesiano/3</code>

Consulta en consola	Resultado esperado
?- <code>cartesiano([a,b],[1,2],R).</code>	$R = [[a,1],[a,2], [b,1],[b,2]].$
?- <code>cartesiano([], [1,2], R).</code>	$R = [].$

Solución: cartesiano/3

Archivo listas.pl

```
cartesiano([],_,[]).  
cartesiano([X|Xs],B,R) :-  
    linea(X,B,Fila),  
    cartesiano(Xs,B,Resto),  
    concat(Fila,Resto,R).
```

Cada X aporta una fila;
concat/3 reúne todas las filas.
Si A tiene m elementos y B
tiene n, obtenemos $m \times n$ pares.

Consulta en consola	Respuesta
?- cartesiano([a,b],[1,2],R).	R = [[a,1],[a,2], [b,1],[b,2]].
?- cartesiano([],[1,2],R).	R = [].